Ten layers, one engine, and nothing crossing the wire but facts.

The architecture ADR-001 described had eleven layers, three of which were AI. The decisions taken since removed the AI entirely and made the client authoritative for every number a shopkeeper sees. What is left is smaller, and the part that matters — the arithmetic — now exists in exactly one specified place and two verified implementations.

REMOVED

~~Layer 4 · Speech Intelligence~~

Whisper, STT, multilingual voice capture. Decision 1 removed AI from the input path entirely.

REMOVED

~~Layer 10 · Knowledge & Learning~~

Proprietary model training on usage data. Nothing to train on, and nothing in V1 that needs it.

DEFERRED

~~Layer 11 · Autonomous Operations~~

AI-driven monitoring and incident response. One instance and a person is the correct answer at this scale.

PROMOTED

Domain Layer

Was one layer among eleven. It is now the centre of the system and the only thing a competitor cannot copy in a weekend.

01 — THE LAYER MODEL

Ten layers, split by the one network hop

Read top to bottom: that is the path a cash entry takes from a shopkeeper's thumb to durable storage. Everything above the dashed line runs on the phone and works with the network switched off.



The two filled layers are the same rules expressed twice. That is deliberate and it is the subject of section 02 — it is also the only duplication in the system that is allowed to exist.

LAYER	OWNS	MUST NEVER DO
ON THE PHONE		
L1	Rendering, tokens, accessibility	Contain a hex literal, compute a total, or decide a business rule.
L2	Routing, form state, focus, origin	Know the arithmetic. It asks L3 and displays the answer.
L3	Formula Engine, day lifecycle, validation	Touch the network, the database, or any widget. Pure functions only — which is what makes it testable and portable.
L4	SQLite, append-only writes, outbox	Store a derived total. Ever. Entries in, entries out.
L5	Outbox drain, retry, token refresh	Block a user action, or let a failed sync surface as an error on an entry screen.
IN MUMBAI		
L6	TLS, DDoS, rate limits, caching	Hold state or terminate business logic.
L7	REST surface, auth, idempotency, tenancy	Compute a figure the client will display, or accept a write without a shop scope.
L8	Independent recomputation, divergence alarm	Silently overwrite a client figure. A mismatch is a bug report, not a correction.
L9	Durable entries, closes, parties, audit	Update or delete an entry row. Append only, forever.
L10	Backups, migrations, monitoring, IaC	Exist only in someone's shell history.

02 — THE CENTRAL DECISION

The engine is specified once and implemented twice

This is the architectural consequence of being offline-first and correctness-first at the same time, and it is the part most likely to be got wrong by a team in a hurry.

Offline-first means the phone must compute every figure without asking anyone — otherwise the app is useless in bad signal, which is most of the time. Correctness-first means something independent has to be able to say the phone got it right. Those two requirements force the same arithmetic to exist in two runtimes, and the naive resolutions are both wrong: sharing code across Dart and the server language creates a build coupling you will regret, and letting them drift creates two truths about a shopkeeper's money.

The resolution: one specification, one test-vector file, two implementations

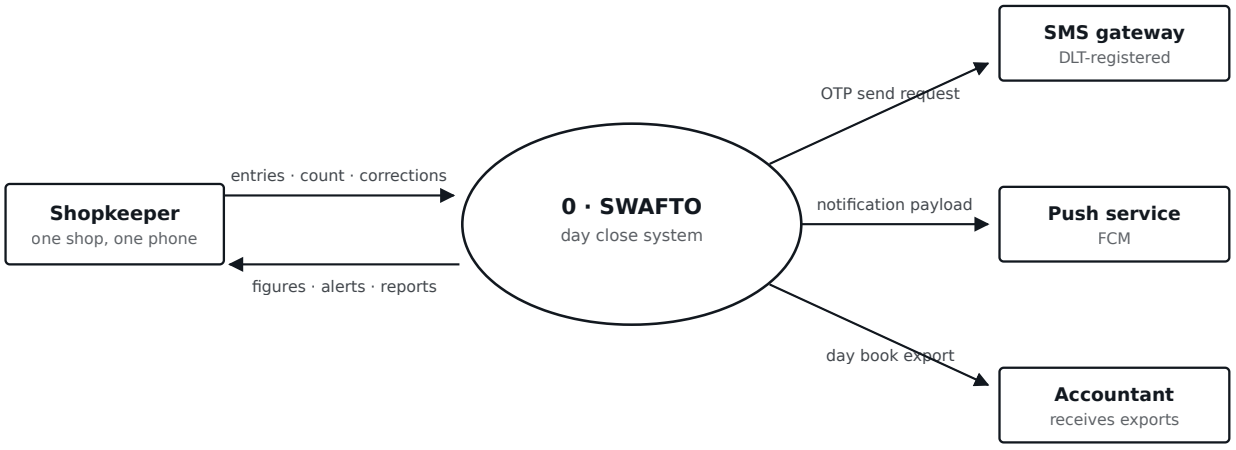
Write the rules once as a plain-text specification with a companion `vectors.json` — a few hundred worked examples, each a set of entries and every figure they should produce. Both implementations import that same file as their test fixture. CI fails if either one disagrees with a vector. You get independent implementations that cannot silently diverge, without coupling two codebases, and the vectors become the artefact a new developer reads to understand the product's actual rules.

RULE UNDER TEST	VECTOR MUST PROVE
`Cash equation`	Only `Cash`-mode expenses and purchases reduce the cash balance; UPI, Other, Udhar and Shop stock do not — while all five still count toward total expenses.
`Withdrawal`	Reduces cash, never appears in expenses, never touches profit.
`Carry-forward`	Day N opening equals day N−1 *counted* closing, never the expected figure.
`Missed day`	A gap produces a `not_closed` day and never an interpolated balance.
`Correction`	An adjustment posted on day N referencing closed day N−3 changes day N's figures and leaves N−3 byte-identical.
`Skip vs empty`	A skipped section and an untouched section produce the same total and different day states.

03 — DATA FLOW, LEVEL 0

Context diagram

What the system exchanges with the world outside it. Four external entities, and notably no payment rail, no bank and no accounting package — V1 talks to nothing that could change a number behind the shopkeeper's back.

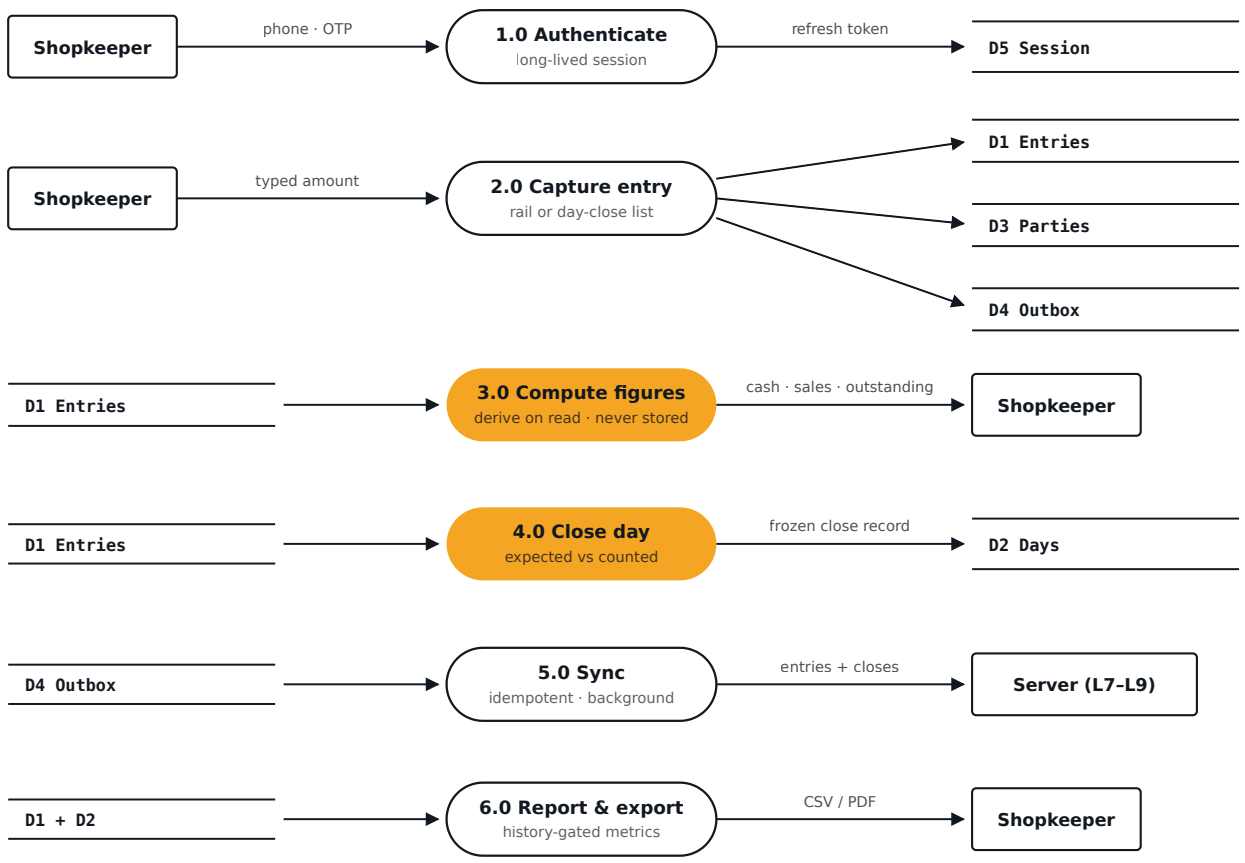


The SMS gateway delivers the code to the handset outside this system's boundary; the shopkeeper then types it back in, which is why the only modelled flow is the send request.

04 — DATA FLOW, LEVEL 1

Six processes, five stores

Each band is one process reading left to right. Stores are shown wherever they are touched, which is conventional and keeps the arrows from crossing.

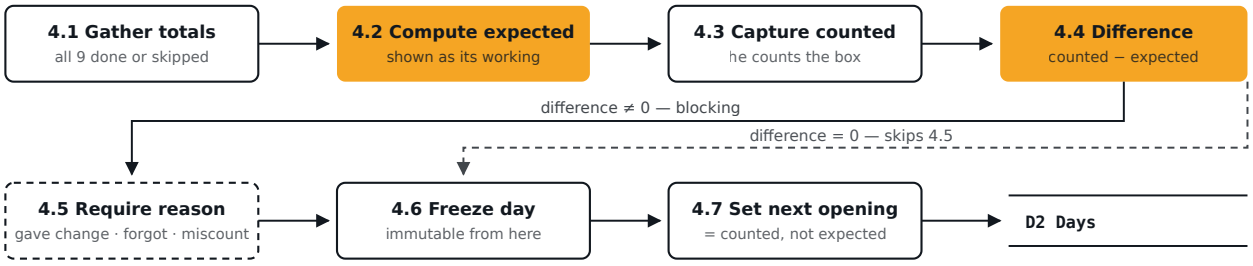


Processes 3.0 and 4.0 are filled because they are the Formula Engine — the only two that turn stored facts into money figures. Every store they read is append-only, so any figure on any screen can be reproduced from the entry log at any later date.

05 — DATA FLOW, LEVEL 2

Process 4.0 — closing the day

Expanded because this is the product. Note the branch at 4.4: the reason step exists only on a non-zero difference, and it is blocking.



4.5 is dashed because it is conditional. It is also the only step in the whole system that blocks the shopkeeper, and that is deliberate: an unexplained difference entered today is an unanswerable question next month.

4.7 is where the audit chain is forged. Because the next day's opening is the counted figure and never the expected one, every subsequent balance inherits a number a human physically verified.

06 — DATA STORES

Five stores, two of them append-only forever

STORE	SHAPE	RULES
D1 Entries	`id · shop_id · day_id · section · amount · mode · party_id · due_date · corrects_day_id · created_at · source`	**Append-only.** An edit writes a new version; a correction writes a new row carrying `corrects_day_id`. `id` is client-generated so the server upsert is idempotent and a retried outbox can never duplicate a sale. `source` is always `manual` in V1 and exists so UPI import stays reversible without a migration.
D2 Days	`shop_id · date · state · opening_cash · counted_cash · difference · reason · closed_at`	One row per shop per date. `state` ∈ open, ready, closed, not_closed. **Immutable once closed.** No totals stored — only the two figures a human supplied and the reason.
D3 Parties	`id · shop_id · name · phone · kind · default_due_days`	Customers and suppliers in one table with a kind flag. Balances are never stored — they derive from D1.
D4 Outbox	`entry_id · attempts · last_error · queued_at`	Local only, never synced. Drains in order; a permanent failure surfaces in the sync detail view, never on an entry screen.
D5 Session	`shop_id · refresh_token · access_token · expires_at`	Device keystore, not the database. Long-lived by decision — a re-auth costs a real SMS.

> **There is no totals table, and there never will be**
>
> Every figure in this document — cash received, expected balance, net sales, a customer's outstanding, a month's profit trend — is computed from D1 and D2 at read time. Caching them is the standard optimisation and the standard way a ledger ends up displaying a number that no longer matches its own entries. At 25 entries a day the computation is free; the staleness would not be.

07 — CROSS-CUTTING RULES

Six invariants that hold at every layer

If a change would break one of these, it is not a change — it is a different product.

01 **The local write is the write**

A save commits to SQLite and the UI proceeds. No spinner, no optimistic-then-rollback, no network in the critical path. Sync is a background consequence, never a precondition.

02 **Every write is idempotent**

Client-generated UUIDs mean the outbox can retry forever without creating a duplicate entry. This is what makes rule 1 safe, and it is the single most commonly skipped piece of offline-first plumbing.

03 **Nothing derived crosses the wire**

The API exchanges entries, day records and parties. It never returns a computed total for display. One engine on the phone produces every number the shopkeeper reads, which is why the app is complete offline including its reports.

04 **Append-only, forever**

No `UPDATE` and no `DELETE` on D1. Corrections are new rows that reference old days. The entry log is the source of truth and every screen is a projection of it.

05 **One shop, one phone**

An explicit non-goal for V1, stated so nobody builds conflict resolution for a case that does not occur. Revisit only when a real shop asks for a second device, and treat it as a genuine architectural change rather than a setting.

06

Divergence is an alarm, not a repair

When the server's recomputation disagrees with the client's, the server does not overwrite. It records both and raises. A money app that silently corrects itself is a money app whose bugs you never find.

---

08 — DEPLOYMENT VIEW

Which layer runs on which box

LAYERS	RUNS ON	NOTES
L1–L5	Android handset, Flutter	Budget devices, Android-first. iOS target exists in the build but is not a launch platform.
L6	Cloudflare free tier	Replaces a load balancer entirely at this scale.
L7–L8	One EC2 t4g.small, public subnet	Docker, systemd, security group. No NAT gateway, no ECS, no Kubernetes.
L9	RDS PostgreSQL db.t4g.micro, single-AZ	7-day automated backups plus a nightly logical dump to S3 — two independent recovery paths.
L10	Terraform, GitHub Actions, CloudWatch	~200 lines of IaC. This is the mitigation for running a single node: rebuild in thirty minutes rather than from memory.

Total infrastructure cost at 500 shops is ₹3,180 a month, and the architecture does not need to change until roughly 15,000 shops — at which point L7 becomes two instances behind a load balancer and L9 steps up an instance class. Nothing else in this document moves.

---

09 — DELIBERATE ABSENCES

What this architecture does not have

Listed because every one of them is something a reviewer will ask about, and each absence is a decision rather than an oversight.

NOT PRESENT	WHY
Any AI or ML in the request path	Decision 1. Also the evidence that manual entry builds a stronger habit than low-burden capture.
A message queue	The outbox on the device is the queue. A server-side broker at 600,000 requests a month is infrastructure looking for a problem.
A cache layer	Nothing is expensive enough to cache, and a cache over derived figures would reintroduce exactly the staleness section 06 exists to prevent.
Microservices	One domain, one team of interns, one deployable. Service boundaries cost more than they save below about ten engineers.
Real-time / websockets	One shop, one phone. There is no second client to push to.
A payment rail or bank integration	V1 records money that has already moved. Nothing in this system can move a rupee, which removes an entire class of risk and regulation.
Item or inventory tables	Decision 5. No input collects them, so no store holds them.

SWAFTO · Retail Desk V1 · final layered architecture and data flow diagrams · 18 September 2026 · supersedes the eleven-layer model in ADR-001 · derived from the nine V1 decisions, the Strategy 1 interaction specification and the infrastructure cost model.